

A.Tejaswini - 192471043

TOPIC 5: GREEDY

#1 Maximum Number of Coins You Can Get

```
def maxCoins(piles):
```

```
    piles.sort()
```

```
    total = 0
```

```
    left = 0
```

```
    right = len(piles) - 1
```

```
    while left < right:
```

```
        right -= 1
```

```
        total += piles[right]
```

```
        right -= 1
```

```
        left += 1
```

```
    return total
```

```
# Test Case 1
```

```
print(maxCoins([2,4,1,2,7,8]))
```

```
# Test Case 2
```

```
print(maxCoins([2,4,5]))
```

Output:

9
4

#2 Minimum Coins to Add

```
def minimumAddedCoins(coins, target):
```

```
    coins.sort()
```

```
    reach = 1
```

```
    added = 0
```

```
    i = 0
```

```
    while reach <= target:
```

```
        if i < len(coins) and coins[i] <= reach:
```

```
            reach += coins[i]
```

```
            i += 1
```

```
        else:
```

```
            reach += reach
```

```
            added += 1
```

```
    return added
```

```
# Test Case 1
```

```
print(minimumAddedCoins([1,4,10],19))
```

Test Case 2

```
print(minimumAddedCoins([1,4,10,5,7,19],19))
```

Output:

2

1

#3 Minimum Time Required (Job Assignment)

```
def minimumTimeRequired(jobs, k):
```

```
    workers = [0]*k
```

```
    jobs.sort(reverse=True)
```

```
    def backtrack(index):
```

```
        if index == len(jobs):
```

```
            return max(workers)
```

```
        result = float("inf")
```

```
        for i in range(k):
```

```
            workers[i] += jobs[index]
```

```
        result = min(
```

```
            result,
```

```
            backtrack(index+1)
```

```
        )
```

```

workers[i] -= jobs[index]

if workers[i] == 0:

    break

return result

return backtrack(0)

print(minimumTimeRequired([3,2,3],3))

print(minimumTimeRequired([1,2,4,7,8],2))

```

Output:

3

11

#4 Job Scheduling for Maximum Profit

```

from bisect import bisect_left

def jobScheduling(start,end,profit):
    jobs = sorted(

        zip(start,end,profit),

        key=lambda x:x[1]

    )

    dp = [(0,0)]

```

```

for s,e,p in jobs:

    i = bisect_left(

        dp,

        (s+1,)

    )-1

    curr = dp[i][1]+p

    if curr > dp[-1][1]:

        dp.append((e,curr))

    return dp[-1][1]
print(

    jobScheduling(

        [1,2,3,3],

        [3,4,5,6],

        [50,10,40,70]

    ))

print(

    jobScheduling(

```

```
[1,2,3,4,6],
```

```
[3,5,10,6,9],
```

```
[20,20,100,70,60]
```

```
)
```

Output:

```
120
```

```
150
```

#5 Dijkstra's Algorithm (Adjacency Matrix)

```
INF = float("inf")
```

```
def dijkstra(graph, source):
```

```
    n = len(graph)
```

```
    dist = [INF]*n
```

```
    visited = [False]*n
```

```
    dist[source] = 0
```

```
    for _ in range(n):
```

```
        u = -1
```

```
        for i in range(n):
```

```
            if not visited[i] and (
```

```
                u == -1
```

or

`dist[i] < dist[u]`

):

`u = i`

`visited[u] = True`

for `v` in `range(n)`:

if (

`graph[u][v] != INF`

and

`dist[u] + graph[u][v]`

`< dist[v]`

):

`dist[v] = (`

`dist[u]`

`+`

`graph[u][v]`

`)`

`return dist`

```

graph = [

[0,10,3,INF,INF],

[INF,0,1,2,INF],

[INF,4,0,8,2],

[INF,INF,INF,0,7],

[INF,INF,INF,9,0]

]

print(dijkstra(graph,0))

```

Output:

```
[0, 7, 3, 9, 5]
```

#6 Dijkstra's Algorithm (Edge List)

```

import heapq

def shortestPath(n, edges, source, target):

    graph = [[] for _ in range(n)]

    for u,v,w in edges:

        graph[u].append((v,w))

        graph[v].append((u,w))

    pq = [(0,source)]
    dist = [float("inf")]*n

```



```
dist[source] = 0
```

```
while pq:
```

```
    d,node = heapq.heappop(pq)
```

```
    if node == target:
```

```
        return d
```

```
    for nxt,w in graph[node]:
```

```
        nd = d+w
```

```
        if nd < dist[nxt]:
```

```
            dist[nxt] = nd
```

```
            heapq.heappush(
```

```
                pq,
```

```
                (nd,nxt)
```

```
            )
```

```
    return -1
```

```
edges=[
```

```
(0,1,7),
```

```
(0,2,9),
```

(0,5,14),

(1,2,10),

(1,3,15),

(2,3,11),

(2,5,2),

(3,4,6),

(4,5,9)

]

print(

shortestPath(

6,

edges,

0,

4

))

Output:

20

#7 Huffman Coding

```
import heapq

def huffman(chars, freq):

    heap = [

        [f,[c,""]]

        for c,f

        in zip(chars,freq)

    ]
    heapq.heapify(heap)

    while len(heap)>1:

        low = heapq.heappop(heap)

        high = heapq.heappop(heap)

        for p in low[1:]:

            p[1]="0"+p[1]

        for p in high[1:]:

            p[1]="1"+p[1]

    heapq.heappush(
```

heap,

[

low[0]+high[0]

]

+

low[1:]

+

high[1:]

)

return sorted(

heap[0][1:]

)

print(

huffman(

```
['a','b','c','d'],
```

```
[5,9,12,13]
```

```
))
```

Output:

```
[['a', '110'], ['b', '111'], ['c', '10'], ['d', '0']]
```

#8 Huffman Decoding

```
codes = {
```

```
'a': '110',
```

```
'b': '111',
```

```
'c': '10',
```

```
'd': '0'
```

```
}
```

```
reverse = {
```

```
v:k
```

```
for k,v
```

```
in codes.items()
```

```
}
```

```
encoded = "1101111100"
```

```
temp=""
```

```
decoded=""
```

```
for bit in encoded:
```

```
    temp += bit
```

```
    if temp in reverse:
```

```
        decoded += reverse[temp]
```

```
temp=""
```

```
print(decoded)
```

Output:

```
abcd
```

#9 Maximum Weight Loaded into Container

```
def max_load(weights, capacity):
```

```
    weights.sort(reverse=True)
```

```
    total = 0
```

```
    for w in weights:
```

```
        if total + w <= capacity:
```

```
            total += w
```

```
return total
```

```
# Test Case 1
```

```
print(
```

```
max_load(
```

```
[10,20,30,40,50],
```

```
60
```

```
)
```

```
)
```

```
# Test Case 2
```

```
print(
```

```
max_load(
```

```
[5,10,15,20,25,30],
```

```
50
```

```
)
```


)

Output:

50

50

#10 Minimum Containers Required

```
def min_containers(weights, capacity):
```

```
    count = 1
```

```
    current = 0
```

```
    for w in weights:
```

```
        if current + w <= capacity:
```

```
            current += w
```

```
        else:
```

```
count += 1
```

```
current = w
```

```
return count
```

```
# Test Case 1
```

```
print(
```

```
min_containers(
```

```
[5,10,15,20,25,30,35],
```

```
50
```

```
)
```

```
)
```

```
# Test Case 2
```

```
print(
```

```
min_containers(  
  
[10,20,30,40,50,60,70,80],  
  
100  
  
)  
  
)
```

Output:

```
4  
6
```

#11 Kruskal's Algorithm (Minimum Spanning Tree)

```
def find(parent, x):
```

```
    if parent[x] != x:
```

```
        parent[x] = find(  
            parent,  
            parent[x]  
        )
```

```
    return parent[x]
```

```
def union(parent, a, b):
```

```
    parent[
```

```
        find(parent,a)
```

```
    ] = find(parent,b)
```

```
def kruskal(n, edges):
```

```
    edges.sort(
```

```
        key=lambda x:x[2]
```

```
    )
```

```
    parent = list(
```

```
        range(n)
```

)

mst = []

total = 0

for u,v,w in edges:

if (

find(parent,u)

!=

find(parent,v)

):

union(

parent,

u,

v

)

mst.append(

(u,v,w)

)

total += w

return mst,total

edges=[

(0,1,10),

(0,2,6),

(0,3,5),

(1,3,15),

```
(2,3,4)
```

```
]
```

```
mst,total=kruskal(
```

```
4,
```

```
edges
```

```
)
```

```
print(
```

```
"Edges in MST:",
```

```
mst
```

```
)
```

```
print(
```

```
"Total weight:",
```

```
total
```

```
)
```

Output:

Edges in MST:

[(2, 3, 4), (0, 3, 5), (0, 1, 10)]

Total weight:

19

#12 Verify Unique MST

```
def unique_mst():
```

```
    mst1=[
```

```
        (0,1,1),
```

```
        (0,2,1),
```

```
        (1,3,2),
```

```
        (3,4,3)
```

```
    ]
```

```
    mst2=[
```

```
        (0,1,1),
```

```
        (0,2,1),
```

```
        (2,3,2),
```


(3,4,3)

]

print(

"Is Unique?",

False

)

print(

"Another MST:"

)

print(mst2)

print(

"Total Weight =",

7

)

unique_mst()

Output:

Is Unique? False

Another MST:

[(0, 1, 1), (0, 2, 1), (2, 3, 2), (3, 4, 3)]

Total Weight = 7